



Angular

Bootcamp Tercera Edición

CRUD con HTTPClient

Módulo 4



Desglose del módulo

1. GET, POST, PUT, DELETE con HttpClient
2. Estados de carga
3. Manejo de errores con catchError
4. Interceptor global de errores
5. Feedback al usuario



1. HTTPClient

¿Qué es?

Es el módulo oficial de Angular para hacer peticiones HTTP. Devuelve Observables

```
import { provideHttpClient } from  
'@angular/common/http';  
  
export const appConfig: ApplicationConfig = {  
  providers: [provideHttpClient()]  
};
```

Los principales verbos HTTP

// GET – obtener datos

```
this.http.get<Usuario[]>('https://api.ejemplo.com/usuarios')
```

// POST – crear recurso

```
this.http.post<Usuario>('/usuarios', { nombre: 'Ana',  
email: 'ana@mail.com' })
```

// PUT – reemplazar recurso completo

```
this.http.put<Usuario>('/usuarios/1', { nombre: 'Ana  
López', email: 'ana@mail.com' })
```

// DELETE – eliminar

```
this.http.delete('/usuarios/1')
```





2. Estados de carga

El componente siempre sabe en qué estado está, y la UI refleja ese estado.

- `loading = false;`
- `error = '';`
- `data = [];`



```
cargarEmployees() {  
  
    this.loading = true;  
    this.error = '';  
  
    this.employeeService.getEmployees().subscribe({  
  
        next: (data) => {  
            this.employees = data;  
            this.loading = false;  
        },  
  
        error: () => {  
            this.error = 'Error al cargar empleados';  
            this.loading = false;  
        }  
  
    });  
}
```



```
@if (loading) {  
    <p>Cargando empleados...</p>  
}  
  
@if (error) {  
    <p class="error">{{ error }}</p>  
    <button (click)="cargarEmployees()">Reintentar</button>  
}  
  
@if (!loading && !error && employees.length === 0) {  
    <p>No hay empleados registrados.</p>  
}  
  
@if (!loading && !error && employees.length > 0) {  
    @for (employee of employees; track employee.id) {  
        ...  
    }  
}
```




3. Manejo de errores con CatchError

Es un operador de RxJS que se coloca dentro del `.pipe()` e intercepta cualquier error que ocurra en el Observable antes de que llegue al `.subscribe()`.

```
import { catchError, throwError } from 'rxjs';
import { HttpResponseResponse } from '@angular/common/http';

getEmployees(): Observable<Employee[]> {
  return this.http.get<Employee[]>(`${this.apiUrl}/employees`).pipe(
    catchError((error: HttpResponseResponse) => {

      return throwError(() => error);
    })
  );
}
```



```
catchError((error: HttpResponse) => {  
    error.status    // 404, 500, 401, 0...  
    error.message   // mensaje técnico  
    error.error     // cuerpo de la respuesta del servidor  
}))
```

```
catchError((error: HttpResponse) => {  
    let mensaje = '';  
  
    switch (error.status) {  
        case 401: mensaje = 'No autorizado';           break;  
        case 404: mensaje = 'Recurso no encontrado';  break;  
        case 500: mensaje = 'Error del servidor';     break;  
        default:  mensaje = 'Error inesperado';  
    }  
  
    return throwError(() => new Error(mensaje));  
}))
```



```
this.employeeService.getEmployees().subscribe({  
  next: (data) => {  
    this.employees = data;  
    this.loading = false;  
  },  
  error: (err) => {  
    this.error = err.message;  
    this.loading = false;  
  }  
});
```



4. Interceptor global de errores

¿Qué es un interceptor?

Es una clase que se ejecuta en cada petición HTTP de toda la app, antes de que llegue al componente. Es como un guardia de seguridad en la puerta — todo pasa por él.

- Sin interceptor: cada servicio maneja sus propios errores por separado.
- Con interceptor: un solo lugar maneja todos los errores de toda la app.



¿Cómo se registra?

```
import { provideHttpClient, withInterceptors } from
  '@angular/common/http';
import { errorInterceptor } from './error.interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(withInterceptors([errorInterceptor]))
  ]
};
```



error.interceptor.ts

```
import { HttpInterceptorFn, HttpResponseError } from
 '@angular/common/http';
import { catchError, throwError } from 'rxjs';

export const errorInterceptor: HttpInterceptorFn =
 (req, next) => {
   return next(req).pipe(
     catchError((error: HttpResponseError) => {
       return throwError(() => error);
     })
   );
};
```



employee.service.ts

```
getEmployees(): Observable<Employee[]> {  
  return  
  this.http.get<Employee[]>(`${this.apiUrl}/employees`).  
  pipe(  
    catchError((error: HttpResponse) => {  
      const mensaje = error.status === 404  
        ? 'La lista de empleados no existe'  
        : 'No se pudo cargar la lista';  
  
      return throwError(() => new Error(mensaje));  
    })  
  );  
}
```



employee.component.ts

```
cargarEmployees() {  
  this.loading = true;  
  this.error = '';  
  
  this.employeeService.getEmployees().subscribe({  
    next: (data) => {  
      this.employees = data;  
      this.loading = false;  
    },  
    error: (err) => {  
      this.error = err.message;  
      this.loading = false;  
    }  
  });  
}
```




Servidor falla



Interceptor - throwError



Servicio – Traduce el error a un texto más legible



Componente – Muestra el mensaje en la UI



5. Feedback al usuario

¿En qué consiste?

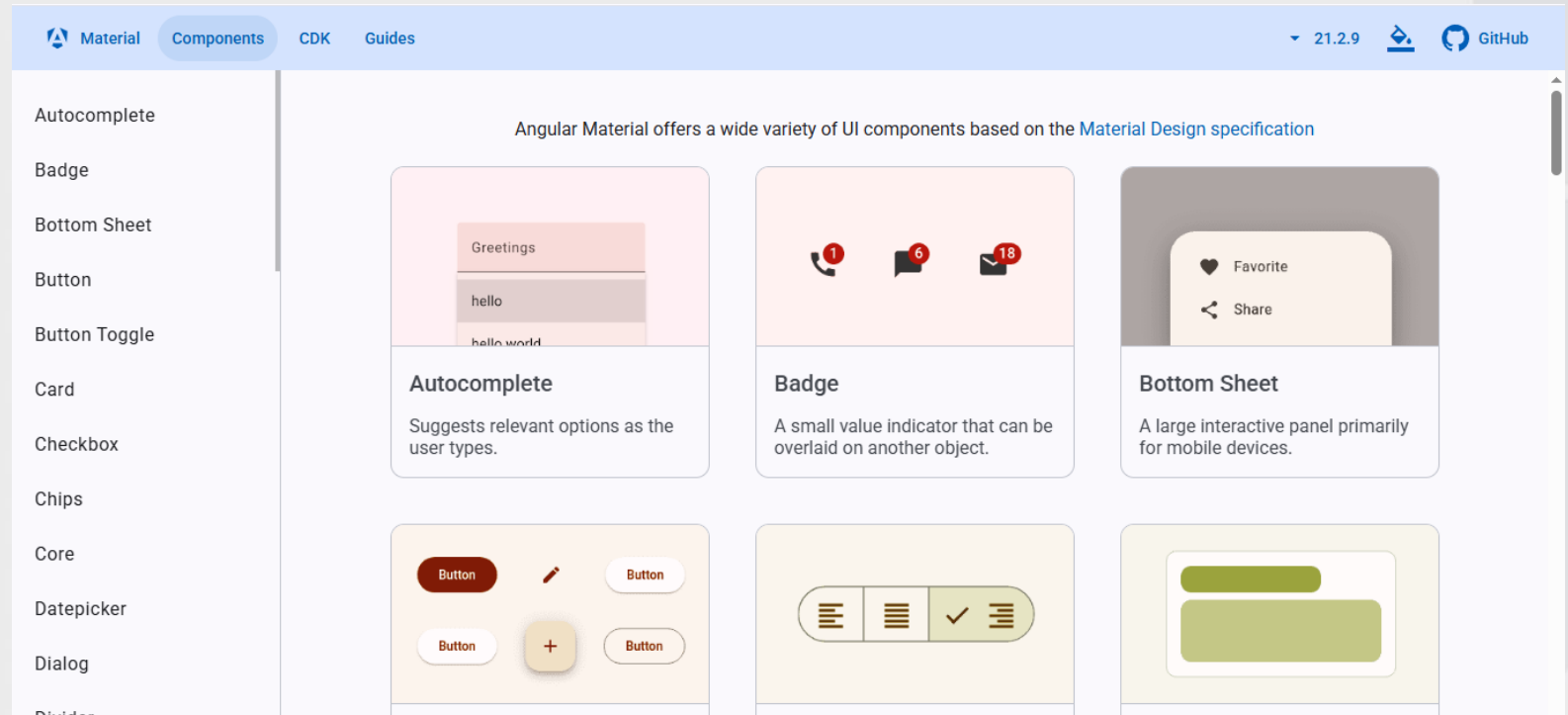
Cualquier mensaje que le dice al usuario qué está pasando o qué pasó

En el contexto de HTTP hay tres momentos:

- mientras espera – loading
- cuando sale bien – mensaje de éxito
- cuando falla – mensaje de error

6. Angular Material

Es la librería oficial de componentes UI para Angular, creada por el mismo equipo de Google.





Para instalarlo:

```
ng add @angular/material
```

```
PS C:\Users\USER\servicios-angular> ng add @angular/material
```

```
✓ Searching for compatible package version
```

```
> Found compatible package version: @angular/material@20.2.14.
```

```
✓ Loading package information from registry
```

```
✓ Confirming installation
```

```
✓ Installing package
```

```
? Select a pair of starter prebuilt color palettes for your Angular Material theme
```

```
> Azure/Blue [Preview: https://material.angular.dev?theme=azure-blue]
```

```
Rose/Red [Preview: https://material.angular.dev?theme=rose-red]
```

```
Magenta/Violet [Preview: https://material.angular.dev?theme=magenta-violet]
```

```
Cyan/Orange [Preview: https://material.angular.dev?theme=cyan-orange]
```



Uso de componentes del material

Cada componente tiene su propio módulo que se importa directamente en el componente a usarse

```
import { MatButtonModule } from  
'@angular/material/button';
```

```
@Component({  
  standalone: true,  
  imports: [MatButtonModule],  
  ...  
})
```



```
<button mat-button>Texto</button>  
<button mat-raised-button>Elevado</button>  
<button mat-flat-button>Plano</button>  
<button mat-stroked-button>Con borde</button>  
<button mat-icon-button>  
  <mat-icon>delete</mat-icon>  
</button>
```

Text	Basic	Disabled	Link
Elevated	Basic	Disabled	Link
Outlined	Basic	Disabled	Link
Filled	Basic	Disabled	Link
Tonal	Basic	Disabled	Link



www.vmt.dev



@vmtdev